

Extended supplement: reporting checklist, pitfalls
checklist,
statistical methods, and measurement details
A Comparability Protocol for Benchmarking Relational Database
Access Layers in Express.js

Mateusz Miotk
University of Gdańsk

This document holds material moved out of the numbered online supplement to keep the journal supplement compact; it is distributed with the replication package. The numbered supplement tables and figures (Supplement Tables S1–S44, Figures S1–S4) remain in `supplement.pdf`.

1 A reporting checklist for access-layer benchmarks

Distilled from the design decisions and the pitfalls this study confronted, the following items are the ones an access-layer benchmark should report so that its claims can be scoped and reproduced. It is a reporting aid, not a standard.

1. **Correctness before timing.** Assert byte-identical responses across layers (final serialized bodies), or state what differs; several defects in this study were caught only this way.
2. **Experimental unit.** Treat the freshly booted run, not the individual request, as the unit; requests within a run are not independent.
3. **Same-host repetition is not environmental replication.** Report how many runs, on how many hosts, over what period; do not call same-host repeats “independent.”
4. **Documentation-selected treatment.** State the exact API, its documentation page and version, the declined alternative, and whether logging, hooks, validation, and identity maps are on.
5. **Latency construct.** Distinguish high-load closed-loop response time (which tracks throughput and queueing) from utilization-controlled or open-loop latency; report the operating point.

6. **Same-SQL / shared-plan control.** Report it as a compound intervention (it changes query formulation, protocol, preparation, round-trips, and mapping together), not a mechanism decomposition.
7. **Resource accounting.** Report application-tier and database CPU separately; throughput per CPU-second, not throughput alone.
8. **Write-state control.** Reset the mutable state between runs and state the durability regime (default versus relaxed).
9. **Provenance.** Pin engine images by digest and dependencies by lock-file; archive raw per-cell records, seeds, and a table-to-record manifest.
10. **Scope.** State the host topology, workload, versions, and dates, and confine claims to them.

2 Benchmarking-pitfalls checklist

The four confounds summarized in the main text’s Discussion, in full. Each is a generic failure mode of the genre, easy to introduce by accident and hard to detect: the correctness ones without an automated cross-check, the performance ones without profiling every cell.

- **Aggregation fan-out.** A naive join between `posts` and `comments` evaluated before the `SUM` multiplies each post’s view count by its number of matching comment rows, inflating the aggregate — a correctness bug invisible unless every layer’s output is checked against a reference, not a performance one.
- **OFFSET vs. keyset pagination.** An early, `OFFSET`-based design for the range-scan endpoint collapsed on MySQL at realistic offsets, which would have been misread as an access-layer weakness rather than what it is: a pagination-strategy cost that InnoDB pays more dearly for than PostgreSQL. Rewriting every adapter around keyset pagination (`WHERE id < cursor ORDER BY id DESC LIMIT n`) removed the confound.
- **Write-workload contamination.** The insert endpoint grows `posts` on every call; because engines and layers commit at different rates, later cells in a run would scan a differently sized table depending on run order rather than on layer identity, unless benchmark-inserted rows are deleted before every cell.
- **Query-formulation confounds.** A first attempt at the aggregation endpoint pre-aggregated *all* of `comments` before filtering to one author, so every request re-scanned the whole table regardless of which author

was requested — a query-plan artifact masquerading as an access-layer effect, fixed by rewriting the query as a correlated subquery touching only the target author’s rows.

3 Statistical methods

This section details the tests summarized in the main text’s Analysis subsection. In every test the unit is the *run-level* statistic (one throughput or p99 value per repeated run, 25 per cell), never the individual request; all confidence intervals are percentile bootstraps resampling replicate indices (preserving the pairing) from a fixed seed (`mulberry32(0x57a75)`), so every resample is reproducible.

To test whether adjacently ranked layers are distinguishable, we compare each adjacent pair on the deep/nested fetch on their per-replicate throughput ratios with a two-sided paired sign-flip permutation test (20,000 permutations, so the smallest attainable p is $1/(20,001)$), cross-checked with the Wilcoxon signed-rank test, and report the geometric-mean paired ratio with a paired bootstrap 95% CI and the paired dominance (the fraction of replicates the faster layer wins). Because p99 is reported at millisecond resolution and contains ties, the permutation test keeps zero log-ratios and the Wilcoxon test drops them with the standard tie correction. Because the seven adjacent pairs follow the *observed* ranking rather than a preregistered one, their significances are a secondary, descriptive robustness check, and we apply a Bonferroni correction as a conservatism check rather than claiming a confirmatory family.

The layer×engine interaction is tested per pattern with a blocked permutation test: for each layer and replicate we form the paired PostgreSQL-minus-MySQL log-throughput difference $D_{\ell,i}$ and, under the null of no interaction, permute the layer labels of $\{D_{\ell,i}\}$ *within* each replicate i (20,000 permutations); the statistic is the between-layer F of a randomized-block ANOVA on D with replicate as the block. Rank agreement across engines (Spearman, Kendall) is reported descriptively over the seven evaluated layers — systems under study, not a sample from a population — so we attach no population confidence intervals to those correlations.

For the pg-Prisma pair we additionally report a paired two-one-sided-tests (TOST) equivalence result against a $\pm 5\%$ margin on the log-ratio. That margin is an author-selected smallest effect of practical interest for *this* study, not a general engineering threshold: at modest deployment scale a sub-5% throughput difference would not change a capacity-planning decision (for example, how many application instances to provision for a target load). We set it from that engineering consequence rather than from measurement noise; it happens also to sit within the run-to-run and day-to-day variation typical of a virtualized host (this study’s own run-to-run CV reaches 15.9% on the

noisiest MySQL-insert cell, within 4.2% on the reads), but that coincidence is not the justification. At very large scale even 5% can correspond to many machines, so the margin should be reset per deployment. We chose it before the analysis but did not preregister it, and the equivalence conclusion is not sensitive to the exact value within a few percent.

4 Measurement details

This section records the measurement specifics summarized in the main text’s Measurement procedure.

Warm-up length The fifteen-second warm-up follows a separate cold-boot measurement for a fast, a middling, and the slowest PostgreSQL layer: every layer reached and sustained at least 95% of steady-state throughput within 12 s (Prisma by 5 s, native by 9 s, MikroORM by 12 s), absorbing JIT compilation, pool fill, and plan caching.

Order invariance and completeness Cell order was reshuffled independently within each replicate; a forward-versus-reversed order check on the write endpoint (5 replicates per direction) found no detectable history effect on the read cells (reversed-to-forward ratio 0.977–1.280, median 1.012; the wide tail is confined to the noisy MySQL inserts, where five replicates cannot separate order from write noise). Of the 450 dedicated write-endpoint boots, 449 completed normally; the exception was one Drizzle/MySQL write replicate whose server boot did not clear the post-reset health check, leaving that cell with 24 of 25 runs (the runner records and skips such failures rather than retrying silently).

Tail estimation `autocannon` records a per-run latency histogram at millisecond resolution; even the slowest deep-fetch cells (≈ 480 – 516 req/s over the 12 s window) complete $\approx 5,800$ – $6,200$ requests per run, so each run-level p99 is estimated from roughly 60 requests in its upper 1% tail, and the reported p99 is the median of the 25 run-level values.

Resource and framework sampling Server CPU and peak resident memory were sampled from `/proc` over the full process tree (parent plus descendants, so an in-process engine or spawned helper cannot escape measurement); a `perf_hooks` GC observer (event count, total pause time) and the 200 ms connection-pool sampler ran inside the server, polled through a `/stats` endpoint after each run. A fixed-payload `/baseline` endpoint returning a seed-realistic thread-shaped payload measured the shared Express-plus-serialization floor: 10,909 (PostgreSQL server) and 10,921 (MySQL server)

req/s, roughly $3\times$ the fastest deep-fetch cell, so the framework floor leaves between-layer spreads visible rather than compressed.

Insert configuration Each insert is an autocommit single-statement `INSERT` under each engine's default isolation level (PostgreSQL Read Committed, MySQL Repeatable Read); the transactional variant wraps a post and its five comments in one transaction. The default durability settings are PostgreSQL `fsync=on`, `synchronous_commit=on`, `full_page_writes=on` and MySQL `innodb_flush_log_at_trx_sync_binlog=1`, `log_bin=ON`.